PixelPine product firmware

Runs Pine on the PY32F003F18P6: the 15 light shows, his face and 18 behaviours, the 6 transitions, the touch star, his brain and his bond, as the simulator (`simulator/index.html`) does. It's a port of the simulator's JavaScript, read side by side: the C names follow the JS ones. Written and brought up on the first rev A board on 2026-11-03.

Status: running on the board. The PC build of the same C code is checked frame by frame against the simulator's own JavaScript (see Matching the simulator). Touch is calibrated on one board and one person. Room-light sensing turned out to be impossible on rev A (see Room light), and a whole-tree touch field was tried and dropped (see Touch field).

Layout

Path	What
`src/shows.c`	the 15 light shows (`ANIMS[]`)
`src/face.c`	face parts, `renderPose()`, the 18 face behaviours (`FB`)
`src/scene.c`	scenes, overlays, the 6 transitions, `goTo()` / `engineTick()`
`src/touch.c`	the gesture classifier (`touchTick()`), unchanged thresholds
`src/brain.c`	emotions and the state machine (`brain`)
`src/pine.c`	one 50 fps frame: touch, brain, engine, gamma 2.2 to 8-bit levels
`src/fx.c`, `src/fx.h`	fixed-point maths: Q12 values, table sin / exp / pow, value noise, the simulator's hashes
`src/jsmath.c`	`Math.sin`, `Math.cos` and `%` bit-identical to V8 (fdlibm, the code V8 uses)
`src/tables.c`, `src/geom.h`	generated by `gen_tables.py` from `hardware/led_map.json`
`hw/`	PY32F003 drivers: startup, clocks, watchdog (`main.c`), BCM charlieplex scan (`scan.c`), touch (`sense.c`), bond in flash (`store.c`)
`host/host_main.c`	PC build for the tests
`test/`	`simref.js` (the simulator, headless), `compare.py` (the comparison), on-board tools

Build and flash

Needs the Arm GNU Toolchain (14.2), Python 3 and, for the tests, Node.js and MSYS2's mingw64 gcc. Flashing needs the J-Link setup in `../flash/`.

```
python gen_tables.py
```

```
python build.py flash
```

Command	Builds
`python build.py`	product firmware, `build/pixelpine.hex`
`python build.py flash`	the same, then programs the board through the J-Link
`python build.py host`	the PC test build, `build/pine_host.exe`
`python build.py showcycle`	bring-up: every show for 8 s in turn, no brain
`python build.py bench`	CPU-time bench; keeps SWD alive (lines L8/L9 dark), read with `test/readbench.py`
`python build.py diag`	LED test patterns, 6 s each: all off / top LED full / top LED dim / all at 64
`python build.py calib`	product firmware with SWD alive, for `test/sensemon.py`, `calibrate.py`, `gestures.py`, `stackcheck.py`

The product firmware turns SWDIO/SWCLK into LED lines L8/L9, so the debugger can only reach it through connect-under-reset (which `flash.py` does).

Resource	Used	Of
Flash	59 296 B (92 %)	63 KB (the last 1 KB holds Pine's bond)
RAM, static	6 896 B	8 KB
Stack, deepest measured	464 B (stack painting, `test/stackcheck.py`)	1 296 B left for it

How it maps onto the hardware

- **Scan:** 16 anode phases x 8 BCM bits, 2 us base slot: 16 x 255 x 2 us = 8.2 ms, **122 Hz** refresh (measured 119.6 Hz with interrupt overhead). This is the simulator's "Raw scan, 8 bit" model: each LED gets round(255 x brightness^2.2). TIM1 interrupts once per bit 2..7; bits 0 and 1 are shown from the phase-start interrupt with a counted delay. Masks are rebuilt every frame, one anode at a time with the interrupt held off.
- **Scan pitfalls found on the board (2026-11-03):**
  - **Pull resistors:** after reset PA13 (SWDIO) has a pull-up and PA14 (SWCLK) a pull-down. As lines L8/L9 they kept ~30 LEDs glowing faintly even with every LED at 0. `scan_init()` turns the pulls off on all 16 lines.
  - **Timer wrap:** if a bit-slot interrupt runs late, the new period can be shorter than the count already reached and the 16-bit timer runs to 65 535 (2.7 ms): bright flashes, worst in the radar. Guarded in the ISR.
  - **Latched phases:** each anode phase copies its 8 masks at its start, so a frame update mid-phase can't mix old and new bits.
  - **Empty slots:** a slot with no LED lit doesn't drive the anode (it would charge every floating line through the anode's LEDs, a faint flash on each).
  - **Current-sharing compensation** (the simulator's "Compensated" model) is in `scan.c` behind `-DSHARE_COMP` but off: on the board, and in the simulator's own model, it made levels jitter frame to frame.
  - **Debug builds never light LEDs:** in the `calib` / `bench` / `listen` builds SWDIO/SWCLK (lines L8/L9) stay on the probe, which holds SWDIO high, so every LED from L8 to a line the scan pulls low lit at full current (two "stuck LED" incidents). Those builds now keep every mask empty. All builds run the watchdog, and the crash handler floats every line first.
  - **Never halt a running board:** a halted core freezes the scan with one slot's pins driven, so one LED gets steady current (~8-9 mA, bright but within the LED's rating). Measure with connect-under-reset or the `calib` / `bench` builds without halting, and use `python build.py diag` for test patterns.
  - **Left over:** at 5 V a tiny current passes through two LEDs in series, so a lit LED can make one other LED glow very softly. Rev B fix: run the board a little below 5 V (a series diode, ~4.3 V).
- **Clock:** internal 24 MHz HSI (no PLL on the F003), 0 flash wait states. SysTick ticks the 50 fps frame.
- **Watchdog:** independent watchdog on the LSI (which has to be switched on first on the PY32), ~1 s, refreshed every frame; product build only, so the debug builds can halt. Checked with a test build (`-DKEEP_SWD -DWDT_TEST`): a deliberate hang from the debugger rebooted Pine after ~1 s.
- **Version:** `"PixelPine PP-240-WW fw 0.1.0"` at 0x080000C0, right after the vector table, readable over SWD.
- **Frame budget:** a frame is 20 ms. Most frames take 4-12 ms; a 205 s run of the normal firmware (calib build) had no late frame and a worst frame of 19.0 ms. A transition between the two heaviest shows with a face overlay can take up to ~24 ms; the main loop then runs the next frame's logic without rebuilding the LED levels, so animation timing stays locked to the simulator's frame count and the LEDs hold their levels for 20 ms. Measured on the board (bench build, 24 MHz):

Show	one frame alone		Show	one frame alone
Twinkle	2.3 ms		Fill & drain	4.1 ms
Snowfall	1.6 ms (+ flakes, ~5 ms full)		Northern lights	8.1 ms
Breathe	1.6 ms		Fireworks	3.8 ms
Spiral garland	4.3 ms		Ornaments	3.7 ms

Show	one frame alone		Show	one frame alone
Candle flicker	8.5 ms		Soft radar	2.9 ms
Star ripple	4.7 ms		Heartbeat	1.9 ms
Fireflies	1.3 ms		Tinsel shimmer	3.3 ms
Rising sparks	2.6 ms		Face overlay / face scene	~1-4 ms

- **Numbers:** per-LED loops run in fixed point (Q12, table sin / exp / pow). The brain, the face behaviours and the per-frame scalars stay in `double`, in the JS's order of operations, so their decisions are the same. Soft double on this core costs ~310 cycles per add or multiply, so particle positions (snow, sparks, fireworks, fireflies) moved to fixed point too.

Matching the simulator

`python test/compare.py` builds the PC version and runs it next to `test/simref.js`, which loads `simulator/index.html` into Node with the page stubbed out. Both get the same seeds, the same random-number stream (`Math.random` replaced by the firmware's xorshift32) and the same touch input (the simulator's recorded gesture signals). What is compared is what the LEDs get: the 8-bit level after gamma 2.2. Results on 2026-11-03:

`Math.sin`, `Math.cos` and `%`: **bit-identical** to V8 on 354 000 arguments.

Whole Pine from power-on (brain + faces + scenes + transitions + touch):

Scenario	Length	State machine	Emotions	LED levels within 1 step
Left alone, 1x clock	120 s	identical every frame	identical (diff 0)	100.00 %
Left alone, 10x clock	180 s	identical every frame (morning > excited > work > daydream > reminded > work > daydream > doze > sleep > morning > work)	identical	99.99 %
Touched a lot (hover, tap, pet, press, squeeze, tickle)	90 s	identical every frame	identical	99.99 %
Dark room, woken up, 30x clock	120 s	identical every frame	identical	100.00 %

Each light show alone, 30 s:

Show	exact	within 1 step		Show	exact	within 1 step
Twinkle	97.0 %	100.00 %		Fill & drain	96.8 %	99.37 %
Snowfall	99.0 %	99.99 %		Northern lights	80.7 %	99.43 %
Breathe	94.5 %	100.00 %		Fireworks	99.8 %	100.00 %
Spiral garland	95.3 %	99.93 %		Ornaments	99.8 %	100.00 %
Candle flicker	85.6 %	100.00 %		Soft radar	96.7 %	100.00 %
Star ripple	98.5 %	100.00 %		Heartbeat	93.8 %	100.00 %
Fireflies	89.0 %	93.31 %		Tinsel shimmer	93.0 %	98.62 %
Rising sparks	99.6 %	99.93 %				

Where it differs, and why:

- **Fireflies** follow the same steering rules, but their paths are chaotic: after ~5 s the fixed-point flies are somewhere else than the simulator's (the first 5 s match 97.5 %). The double-precision version matched to the end but cost 17 ms a frame.
- **Snowfall**: about once in 30 s a flake lands one frame earlier or on the neighbouring column (fixed-point flake positions), so that LED of the snow pile differs until it melts.

- **Northern lights, candle, tinsel, fill**: Q12 rounding in long chains of noise and powers; never more than 5 steps of 255.
- The simulator opens mid-yawn (it pre-runs 175 frames for the preview); the tree starts the yawn from the beginning.
- Pine's clock is the simulator's "1x like the real tree".

Touch calibration

Charge sharing between the star (PA7 through R19, 1 kOhm) and the ADC's sample capacitor. A touching finger also brings mains hum: the board's USB supply floats relative to you, and sampled at exactly 50 Hz the hum turns into a slow beat of several volts. So one sample is taken at every anode change of the scan (~39 per 20 ms frame) and the frame uses their mean. Two PY32 details: the ADC clears ADEN after every conversion (re-enabled per conversion), and ADEN is ignored for a few clocks after calibration.

Measured on the first board (`test/calibrate.py`, 2026-11-03, mean reading per frame, x8):

Gesture	Reading	Share of untouched
Untouched	8 510 +- 15	1.00
Finger resting on the front	~3 500-3 900	~0.44
Firm press, flattened fingertip	~2 200-2 500	~0.27
Pinch through front and back	~1 650-1 800	~0.20
Finger 1 cm above	no change	1.00

`hw/sense.c` maps the ratio piecewise-linearly onto the simulator's scale, with its thresholds halfway between these (touch 0.35 at a 18 % drop, press 0.65 at 0.35, squeeze 1.0 at 0.235). The touch threshold is low on purpose: contact takes a few hundred ms to build up, and a quick tap still has to count. The classifier is unchanged. A gesture test (`test/gestures.py`) recognised tap, tickle, pet and squeeze. A press has to be really firm, and a pinch passes through the press band on its way, so "press" fires just before "squeeze" (the simulator's recorded squeeze jumps straight to its level). A baseline tracker follows drift (~10 s), and something resting on the star for 30 s becomes the new baseline.

Hover can't be sensed with this electrode: a finger 1 cm away changes the reading by less than the noise. The gesture stays in the firmware (it fires if a finger comes very close).

Room light

The simulator's "room lights off, Pine gets sleepy" needs the tree to see the room light. The plan was to use the LEDs as photodiodes, but that **can't work in a full charlieplex matrix**: every pair of lines has two LEDs, one each way round. Reverse-biasing one LED forward-biases its partner, which clamps the line (that is what the first attempt measured: the same ~2.1 V with lights on and off), and without bias the two LEDs' photocurrents cancel. So rev A keeps the simulator's default, lights on: Pine still gets tired, dozes and sleeps on his own energy.

Fix for rev B, at no cost: leave one LED unpopulated (for example the L1 -> L0 one), so its partner L0 -> L1 has no anti-parallel twin and works as a photodiode between frames. The tree loses one pixel.

Touch field (tried, dropped)

The idea: use the 240-LED matrix itself as a touch surface, so a fingertip anywhere on the tree counts (Pine would look toward it, and a tap during a light show would send a ripple out from the finger). Tried on the board 2026-11-04 in five experiments and three runs of a full driver, then dropped: **the signal depends on how the tree is powered**, and on the supplies a customer would actually use it is too weak to be reliable.

Run	Method	Result
1	Lines as capacitive electrodes (charge sharing, RC time)	A fingertip changes a line's ~100-200 pF by <= 0.4 %: lost in the noise

Run	Method	Result
2	Listen for the mains hum a finger brings, every other line at 0 V	Strong and position-dependent: the finger moves the lines it sits on
3	9 spots; digital inputs vs ADC	Digital inputs never see it (the hum stays below the input threshold); the ADC separates 7 of 9 spots
4	Short listening gaps while Pine runs	Nothing: the lines still carry charge from the scan, which swamps the finger. 8 ms gaps flicker visibly
5	**Drain first**: every line to 0 V for 5 us, then float L0-L6 (analog mode, the lightest load) 20 us-2 ms	All 9 spots stand out, each with its own line pattern, rest reading flat (noise < 8 counts); 0.2 ms of float is enough. Board on a laptop's USB port
Driver	Drain gaps in the scan interrupt (2 per refresh, 0.23 ms, ~5 % of LED time), pattern match + interpolation	Powerbank: **no signal at all**. Wall charger: total rise 10-60 counts in one run, 7-14 in the next, against ~1 at rest; 3-4 of 9 spots placed right

Why it depends on the supply. The lines don't sense the finger's capacitance (run 1); they sense the 50 Hz hum a person carries, measured against the board's ground. On a laptop the board's ground is tied to mains and the difference is hundreds of counts. A USB wall charger links its output to mains only through a small capacitor, so the difference is weaker and changes with the charger and with how the person sits. On a powerbank the board floats along with the person and there is no difference to measure.

Why not keep it as an extra. At 7-14 counts a touch is only 3-7x the rest noise: a threshold low enough to catch it is close to false touches (a wrong rest level made the tree "touched" with no hand on it in the last run), and weak patterns are too flat to tell the spots apart. Nothing on rev A raises the signal much: a 2 ms float gives ~2x and costs visible LED time. A feature that works on some chargers and not others would read as a fault.

Found along the way:

- The PY32F003 has two comparators (COMP1 on PA1 = L1, COMP2 on PB6 = L11, against 1/4 Vrefint ~0.3 V). They do see the hum at a ~0.3 V threshold (experiment 5), but add nothing the ADC doesn't give.
- With the probe attached, SWDIO (line L8) is held high: any build that pulls lines low lights the L8 LEDs through the probe. The experiments ran with SWDIO / SWCLK unplugged and read their results once at the end.
- Polling the probe while the board runs pulsed RESET through the probe's wire and restarted the chip.

Rev B: a whole-tree touch would need dedicated electrodes (a copper pad or ring per zone) on free ADC pins, which the 20-pin PY32F003 doesn't have, or a self-capacitance touch IC. The touch star stays the control: it measures capacitance, so it works on any supply. The experiment code is in git history (commits `21a415c`, `b18460c`, `d70cac7`, `6f3f453`, `cee5f76`), and was removed from the tree afterwards.

Pine's bond

Saved to the last 1 KB of flash, an 8-page ring (128-byte pages; each save erases the next page and writes {magic, sequence, bond}). The simulator saves every 5 s; the tree saves when the bond moved by 0.02 or more, at most every 15 minutes, to spare the flash. Checked on the board: three saves rotated through pages 2-4 with rising sequence numbers, and after a reset Pine came back with the stored bond (0.392 instead of the default 0.3). The LEDs go dark for the ~4 ms page erase.

Tests

Command	Checks	Needs
`python test/compare.py`	firmware vs simulator, as above (~5 min)	Node.js, mingw64 gcc
`python test/readbench.py`	CPU time per show on the board	the `bench` build flashed
`python test/stackcheck.py`	deepest stack use	the `calib` build flashed
`python test/sensemon.py [s]`	live touch reading, Pine's state, late frames	the `calib` build flashed
`python test/calibrate.py`	guided touch calibration (spoken steps)	the `calib` build flashed

Command	Checks	Needs
`python test/gestures.py`	guided gesture recognition test	the `calib` build flashed

Not done

- Touch thresholds are from one board and one person; a production batch needs a spread measurement.
- The EMC worst-case firmware mode and the production test firmware (technical file) are separate builds still to write.